

Beyond Strict Rules: Code Smell Detection with LLMs

Assessing the Effectiveness of Large Language Models vs. Static Analysis Tools

Saymon da Silva Souza

PhD student | Computer Science

The Core Challenge of Code Smells

The Old Paradigm



Static Analysis Tools

Strict Rules

Rely on rigid metrics (e.g., lines of code, coupling counts).

The Flaw

Often miss the subjective, semantic context of real-world development, leading to false positives and poor developer agreement.

The New Paradigm



Large Language Models

Flexible Context

Can parse complex logic, naming conventions, and intent.

The Question

Can they reliably navigate massive codebases to identify nuanced smells better than specialized tools?

Establishing the Ground Truth

(Section 3)

30

Top-starred Java
projects from GitHub.

268

Code smell candidates
sampled across 9
distinct smell types.

76 

Software developers
engaged for manual
inspection.

758

Independent manual
evaluations to establish
a baseline of human
perspective.

Raw Project Data

Human-Validated Ground Truth

The Investigators: Selecting the Models

			
GPT-5 mini	Llama-3.3	DeepSeek-R1	Qwen2.5-Coder
128K Context Window	128K Window 70B Params	131K Window 32.5B Params	131K Window 32.5B Params
Closed Source (May 2024)	Open Source (Dec 2023)	Open Source (July 2024)	Open Source (Mar 2024)

Teaching LLMs to Detect: Chain-of-Thought Prompting

1. Definition

Base the model's understanding on literature (e.g., Lanza & Marinescu).



2. Guiding Questions

Force step-by-step reasoning before concluding.



3. Final Conclusion

Standardize output for analysis.

- 1** You are a software expert analyzing a Java file for the Large Class code smell...
- 2** Does the class centralize too much functionality?
Does it heavily access data from other classes?
- 3** Start your answer with "YES, I found Large Class" or "NO, I did not..."

The Roadmap for Investigation (Section 4)

01

RQ1 – LLM Effectiveness

How accurately do LLMs identify code smells relative to the human-validated ground truth?

02

RQ2 – LLMs vs. Static Tools

Where do LLMs offer an advantage, and where do traditional static analysis tools hold the line?

03

RQ3 – The Power of Combination

Can an ensemble voting strategy (LLMs + Tools) maximize detection coverage?

The Combined Ensemble Strategy



Threshold: ≥ 3 Votes = Positive Detection

All votes carry equal weight. The goal is to leverage both the strict rules of tools and the contextual reasoning of LLMs.

RQ1: Where LLMs Shine and Where They Struggle

High F1 > 0.80

Structurally Simple Smells

Smells	Why
Large Class, Long Method, Data Class.	Based on clear patterns like size, class structure, and strong connections.

- **Data Point:** Llama-3.3 hits 0.95 Precision on Large Class; GPT-5 mini hits 0.95 Recall on Long Method.

Low F1 < 0.40

Subjective & Contextual Smells

Smells	Why
Refused Bequest, Shotgun Surgery.	Requires deep, holistic reasoning across scattered modules and inheritance hierarchies.

- **Data Point:** All models score below 0.40 F1 for Refused Bequest.

RQ2: LLMs vs. Static Analysis Tools

Large Language Models

Victorious in 5 out of 9 Smells

[✓] Data Class

[✓] Feature Envy

[✓] Intensive Coupling

[✓] Large Class

[✓] Long Method



Insight: DeepSeek-R1 and Llama-3.3 consistently match or beat tools for localized, modular design issues.

VS

Static Analysis Tools

Victorious in 4 out of 9 Smells

[✓] Dispersed Coupling

[✓] Long Parameter List

[✓] Refused Bequest

[✓] Shotgun Surgery



Insight: Tools like JSplRIT and Organic are still superior for complex semantic tracking and scattered patterns.

RQ3: The Combined Strategy Trade-off

The Benefit:

- Combined prediction achieves the best Recall across 7 out of 9 code smells.
- Hits near-perfect Recall (1.00) for Large Class and Data Class.

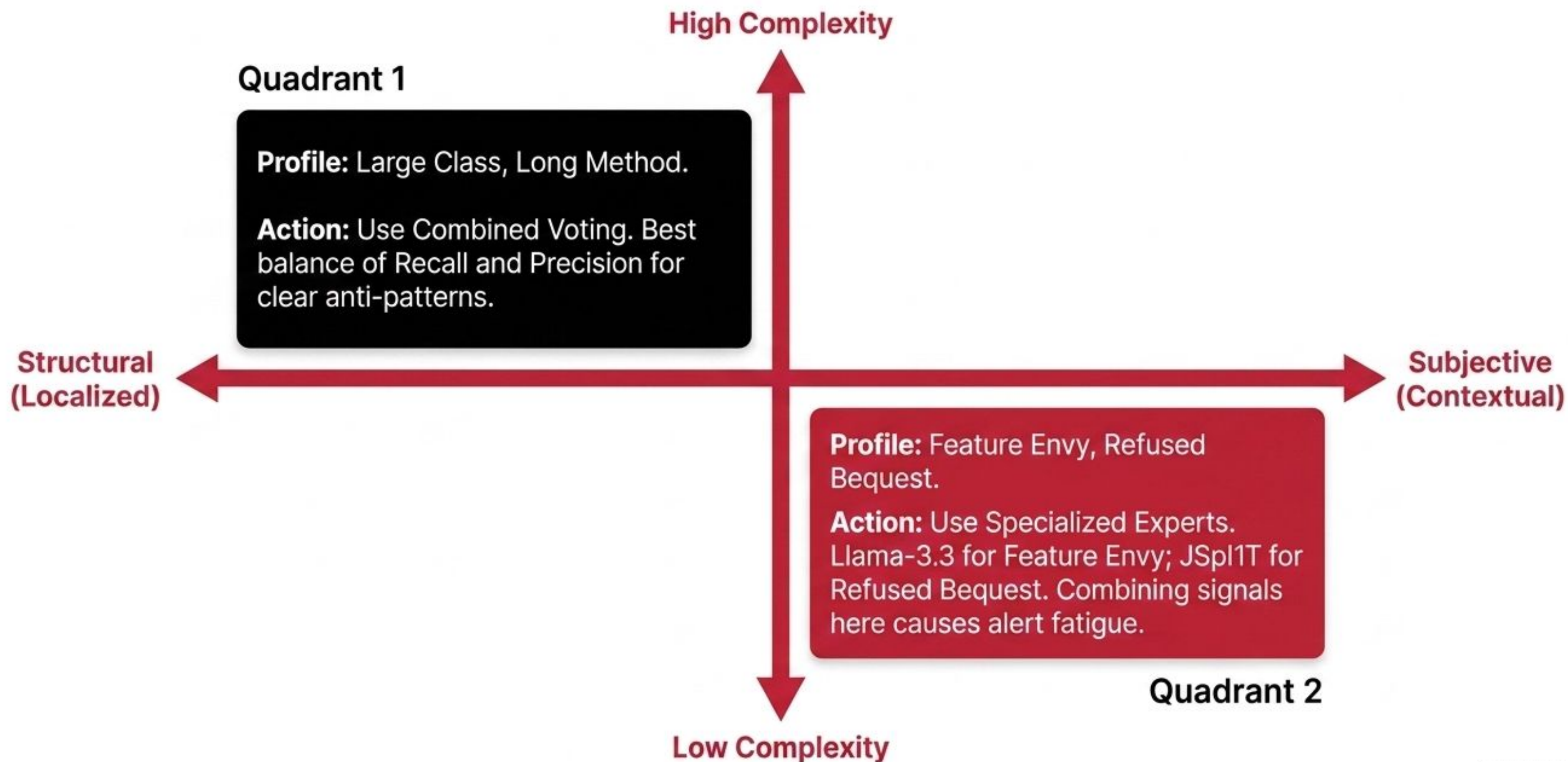
MAXIMUM
RECALL

FALSE
POSITIVES

The Cost:

- Precision Drop on Complex Smells.
- For subjective smells (**Feature Envy**, **Refused Bequest**), combining models **dilutes accuracy**, generating more **false positives** than specialized single tools.

The Synthesis: Choosing the Right Strategy



Practitioner Guide: Precision vs. Recall

**What is your
project's priority?**

We cannot miss any hidden issues.

Prioritize Recall

Context: Legacy systems, safety-critical software.

Strategy: Deploy the Combined Ensemble Strategy. Accept higher manual review costs to ensure no smell is missed.

We want to avoid alert fatigue.

Prioritize Precision

Context: Fast-moving agile teams, limited bandwidth.

Strategy: Deploy Targeted Single Tools (e.g., Organic for Long Parameter List) or tuned LLMs to minimize false positives.

Implications for the Future

Advancing Explainable AI

Leveraging the 'chain-of-thought' dataset to build models that don't just flag issues, but justify their reasoning to developers.

Contextual Prompting & Hybrid Models

Bridging the gap on subjective smells (like Refused Bequest) by feeding LLMs richer design documentation and inheritance hierarchies.

Cross-Domain Evaluation

Adapting the open-source pipeline to evaluate if these LLM capabilities transfer to other programming languages and frameworks beyond Java.



Thank you!

Saymon da Silva Souza
PhD student | Computer Science

